

Contents

Series Foreword	xi
Preface	xiii
1 Introduction	1
1.1 Number Theory and Complexity	3
1.2 Number Theory and Computation: A Brief History	4
1.3 Condensed History of the Theory of Computation	11
1.4 Notes on Chapter 1	13
2 Fundamentals of Number Theory	19
2.1 Notation, Definitions, and Some Computational Problems	19
2.2 More Definitions	22
2.3 Multiplicative Functions and Möbius Inversion	23
2.4 Notation: Big-O, Little-o, Big-Omega, Big-Theta	25
2.5 Abel's Identity and Euler's Summation Formula	25
2.6 Asymptotic Integration	27
2.7 Estimating Sums over Primes	28
2.8 Basic Concepts of Abstract Algebra	29
2.9 Exercises	34
2.10 Notes on Chapter 2	37
3 A Survey of Complexity Theory	41
3.1 Notation	41
3.2 The Notion of "Step"	41
3.3 Language Classes	44
3.4 Reductions and $\mathcal{NP}$ -Completeness	47
3.5 Randomized Complexity Classes	50
3.6 A Formal Computational Model	52
3.7 Other Resources	55
3.8 Parallel Complexity Classes	57
3.9 Exercises	59
3.10 Notes on Chapter 3	63
4 The Greatest Common Divisor	67
4.1 The Euclidean Algorithm	67
4.2 The Euclidean Algorithm: Worst-Case Analysis	68
4.3 The Extended Euclidean Algorithm	70
4.4 The Euclidean Algorithm and Continuants	73

4.5	Continued Fractions	75
4.6	The Least-Remainder Euclidean Algorithm	79
4.7	The Binary gcd Algorithm	82
4.8	Constructing a gcd-Free Basis	84
4.9	Exercises	90
4.10	Notes on Chapter 4	96
5	Computing in $\mathbb{Z}/(n)$	101
5.1	Basics	101
5.2	Addition, Subtraction, Multiplication	101
5.3	Multiplicative Inverse	102
5.4	The Power Algorithm	102
5.5	The Chinese Remainder Theorem	104
5.6	The Multiplicative Structure of $(\mathbb{Z}/(n))^*$	108
5.7	Quadratic Residues	109
5.8	The Legendre Symbol	110
5.9	The Jacobi Symbol	111
5.10	Exercises	114
5.11	Notes on Chapter 5	120
6	Finite Fields	125
6.1	Basics	125
6.2	The Euclidean Algorithm	127
6.3	Continued Fractions	130
6.4	Computing in $k[X]/(f)$	132
6.5	Galois Theory	133
6.6	The Structure of $k[X]/(f)$	136
6.7	Characters	141
6.8	Exercises	143
6.9	Notes on Chapter 6	148
7	Solving Equations over Finite Fields	155
7.1	Square Roots: Group-Theoretic Methods	155
7.2	Square Roots: Field-Theoretic Methods	157
7.3	Computing d -th Roots	160
7.4	Polynomial Factoring Algorithms	163
7.5	Other Results on Polynomial Factoring	168
7.6	Synthesis of Finite Fields	171

7.7	Hensel's Lemma	
7.8	Complexity-Theoretic Results	173
7.9	Exercises	177
7.10	Notes on Chapter 7	188
8	Prime Numbers: Facts and Heuristics	194
8.1	Some History	203
8.2	The Density of Primes	204
8.3	Sharp Estimates and the Riemann Hypothesis	206
8.4	Primes in Arithmetic Progressions and the ERH	211
8.5	Applications of the ERH	215
8.6	Other Conjectures about Primes	217
8.7	Extensions to Algebraic Numbers	224
8.8	Some Useful Explicit Estimates	227
8.9	Exercises	233
8.10	Notes on Chapter 8	236
9	Prime Numbers: Basic Algorithms	245
9.1	Primality Proofs and Fermat's Theorem	265
9.2	Primality Tests for Numbers of Special Forms	266
9.3	Pseudoprimes and Carmichael Numbers	272
9.4	Probabilistic Primality Tests	275
9.5	ERH-Based Methods	278
9.6	Primality Testing Using Algebraic Number Theory	283
9.7	Generation of "Random" Primes	285
9.8	Prime Number Sieves	293
9.9	Computing $\pi(x)$ and p_n	295
9.10	Exercises	299
9.11	Notes on Chapter 9	303
A	Solutions to Exercises	308
	Bibliography	319
	Index to Notation	389
	Index	487
		489

Series Foreword

Theoretical computer science has now undergone several decades of development. The "classical" topics of automata theory, formal languages, and computational complexity have become firmly established, and their importance to other theoretical work and to practice is widely recognized. Stimulated by technological advances, theoreticians have been rapidly expanding the areas under study, and the time delay between theoretical progress and its practical impact has been decreasing dramatically. Much publicity has been given recently to breakthroughs in cryptography and linear programming, and steady progress is being made on programming language semantics, computational geometry, and efficient data structures. Newer, more speculative, areas of study include relational databases, VLSI theory, and parallel and distributed computation. As this list of topics continues expanding, it is becoming more and more difficult to stay abreast of the progress that is being made and increasingly important that the most significant work be distilled and communicated in a manner that will facilitate further research and application of this work. By publishing comprehensive books and specialized monographs on the theoretical aspects of computer science, the series on Foundations of Computing provides a forum in which important research topics can be presented in their entirety and placed in perspective for researchers, students, and practitioners alike.

Michael R. Garey
Albert R. Meyer

Preface

This is the first volume of a projected two-volume set on algorithmic number theory, the design and analysis of algorithms for problems from the theory of numbers. This volume focuses primarily on those problems from number theory that admit relatively efficient solutions. The second volume will largely focus on problems for which efficient algorithms are not known, and applications thereof.

Prerequisites

We hope that the material in this book will be useful for readers at many levels, from the beginning graduate student to experts in the area. The early chapters assume that the reader is familiar with the topics in an undergraduate algebra course: groups, rings, and fields. Later chapters assume some familiarity with Galois theory. A good text is Herstein [1975].

We assume the reader is familiar with the analysis of algorithms, as treated, for example, in Aho, Hopcroft, and Ullman [1974], or Cormen, Leiserson, and Rivest [1990] and with the language of complexity theory, as treated, for example, in Garey and Johnson [1979]. However, we have tried to make our discussion of complexity theory relatively self-contained.

Finally, we assume the reader has had the equivalent of an undergraduate course in probability theory. A good text in this area is Feller [1968].

For some sections, a knowledge of more advanced mathematics will be useful. For example, complex analysis is needed for parts of Chapter 8, and some algebraic number theory will be useful for Chapter 9.

A typical graduate course for computer scientists might cover the following material: Chapter 1, Chapter 2, Chapter 4 (Sections 1–3), Chapter 5, Chapter 6 (Sections 1–5), Chapter 7 (Sections 1–4), Chapter 8 (Sections 1–5), Chapter 9 (Sections 1–5), and Chapters 11 and 12. Mathematicians may want to substitute Chapter 3 for Chapter 2.

Goals of This Book

As stated above, this book discusses the current state of the art in algorithmic number theory. This book is *not* an elementary number theory textbook, and so we frequently do not give detailed proofs of results whose central focus is not computational. Choosing otherwise would have made this book twice as long.

However, we firmly believe that a mere list of proved theorems is not particularly enlightening or useful. Therefore, we endeavor to prove as much as is feasible. Some proofs are left to the reader as exercises, with outlines suggested in Appendix A. And every

theorem which is not proved in the text or left as an exercise has a reference in the “Notes” section that appears at the end of each chapter.

This book is also not intended solely as a practical guide for those who wish to implement number-theoretic algorithms. The variety of architectures of modern machines, and the profusion of languages and operating systems, frequently make specific remarks on running times useless. On the other hand, computational theory without the influence of practice seems artificial. Thus, the “Notes” section of each chapter contains remarks on practical implementations of the algorithms discussed.

We might mention several other books that cover approximately the same material. The first was Knuth [1969]; a second edition appeared as Knuth [1981]. More recently, there have appeared texts such as Riesel [1985a]; Kranakis [1986]; Koblitz [1987a]; Bressoud [1989]; H. Cohen [1993]; and Zippel [1993]. We hope that our book will be useful in addition to these, because of its somewhat different emphasis.

We restrict our attention in this book to those concepts and algorithms that relate to what we see as algorithmic number theory. Thus, we do not discuss some very interesting topics that seem more algebraic than number-theoretic: algorithms on finite groups, Gröbner bases, etc. We also restrict our attention to problems from *elementary* number theory: e.g., primality testing, factorization, discrete logarithm, etc. Thus there is virtually no overlap between the present work and the books of Zimmer [1972] and Pohst and Zassenhaus [1989].

In writing this book, we have found many opportunities to extend or refine known results. This is particularly true with regard to algorithms, many of whose running times were not completely worked out in the literature. As an alternative to listing all such improvements (which is clearly impractical), we give references to earlier results, such as are known to us, in the appropriate places.

Algorithm Descriptions

We write our algorithms in a “pseudo-code” notation similar to Pascal or C, which should be familiar to most readers. We do not provide a formal definition of this notation.

We do make the following observations: no explicit “begin/end” is used to denote the start and finish of a loop. Instead, the scope of loops is made clear through indentation. A “return” statement causes execution of a routine to terminate immediately. An “output” statement is identical to “return”, except that execution continues with the next line; this allows a program to return multiple outputs at different places in the program.

Projected Contents of Volume II

Volume II: Intractable Problems will cover the following topics:

- Chapter 10** *Solving Equations*
- Chapter 11** *Factoring Integers*
- Chapter 12** *Discrete Logarithms*
- Chapter 13** *Applications of Algebraic Geometry*
- Chapter 14** *Reductions Among Number-Theoretic Problems*
- Chapter 15** *Applications*
- Chapter 16** *Open Problems*

Acknowledgments

We would like to thank our thesis advisor, Manuel Blum, for having introduced us to algorithmic number theory and its applications.

Richard Carnes typed the course notes on which an earlier version of this book was based. Terrence Dick provided bibliographic and programming assistance.

We express our thanks to the staff members of the Mathematics Faculty Computing Facility at the University of Waterloo (in particular Anil Goel) for explaining some of the arcane features of the software used to write this book.

Thanks to Igor Khurgin, Arkady Kanevsky, and Valery Soloviev for help with Russian translation, to Jean-Paul Allouche for help with French translation, and to Christiane Whittington and Renate Scheidler for help with German translation.

Thanks to Dan Shanks for providing the reference to Robinson's factor stencils.

Thanks to Andrei Brodnik for bringing our attention to articles on maintaining partial sums of an array.

We would like to express our appreciation to the following people who read a preliminary version of our book and made many comments: John Brillhart, Gudmund Frandsen, Jerry Kuch, Arjen Lenstra, Hendrik W. Lenstra, Jr., Paul Beame, Kevin McCurley, Peter Montgomery, Evi Nemeth, Michael Somos, Rob Lambert, Stan Wagon, Andrew Odlyzko, Alfred Menezes, Louxin Zhang, Drew Vandeth, Will Gilbert, Joan Boyar, Donald Knuth, and Peter Høyer.

We are extremely grateful to many colleagues who have unselfishly given their time locating difficult-to-find papers and books for us. These include Stuart Kurtz, Howard Karloff, Hugh Williams, John Brillhart, and Jean-Paul Allouche. We wish to thank the library staff at the University of Waterloo, the University of Wisconsin, and Dartmouth College for their help. We also thank John Neu, Bibliographer in the History of Science department at the University of Wisconsin.

Of course, any mistakes are our responsibility alone. Please report mistakes by sending electronic mail to shallit@graceland.uwaterloo.ca or bach@cs.wisc.edu. Errata for the book can be found at

<http://math.uwaterloo.ca/~shallit>

which is the URL for Shallit's home page.

This work was supported in part by grants from the National Science Foundation (USA); Wisconsin Alumni Research Foundation, and the Natural Sciences and Engineering Research Council (Canada).

Eric Bach
Madison, Wisconsin

Jeffrey Shallit
Waterloo, Ontario

1 Introduction

The goal of this book is to provide a thorough introduction to the design and analysis of algorithms for problems from the theory of numbers.

The goal of algorithmic number theory is to *produce efficiently* the objects whose mere *existence* is proved in the classical theorems of number theory. For example, the famous proof of Euclid tells us that there exists a prime larger than any given number n ; algorithmic number theorists ask: how can we quickly find such a prime? The fundamental theorem of arithmetic tells us that every number n can be written as the product of prime powers; algorithmic number theorists ask: can we quickly find this prime factorization of n ? Even to state these questions precisely, we must introduce concepts from the theory of computation. We might say that algorithmic number theory is the marriage of an ancient subject, number theory, with a modern one: the theory of computational complexity.

Almost every problem we will discuss can be interpreted as finding the solution to an equation in a specified group, ring, or field. For example, factoring n can be viewed as finding the integer solutions to the equation $n = xy$, with $x, y > 1$. Thus the discipline of algorithmic number theory can be summarized succinctly as *finding solutions to equations, or proving their non-existence, making efficient use of resources such as time and space*. Implicitly included in this definition is the problem of *efficiently representing* the objects in question on a computer.

Unlike some other branches of mathematics, number theory has long been associated with computation. Roughly speaking, this association may be categorized as follows:

1. *Construction of tables:* Here the investigator may be concerned with obtaining evidence for conjectures. Many tables, computed by hand, can be found in the collected works of Gauss. (He conjectured the prime number theorem on the basis of a table of primes.) Tables giving the prime factorization of integers were constructed to higher and higher limits. To list just a few such tables, Cataldi published a table up to the limit of 800 in 1603, T. Brancker reached 100,000 in 1668, L. Chernac reached 1,020,000 in 1811, and D. N. Lehmer published a table up to 10,017,000 in 1909. Many investigators have constructed tables of periods of continued fractions for square roots, class numbers of algebraic number fields, etc. The celebrated conjectures of Birch and Swinnerton-Dyer, which relate the rank of an elliptic curve with the orders of the zeroes of certain L -functions, were based on extensive computations, as were the more recent heuristics on class groups of H. Cohen and H. W. Lenstra, Jr. Neither of these conjectures have been proved in their full generality.
2. *Gathering evidence for conjectures:* Goldbach, in a 1742 letter to Euler, conjectured that every even integer ≥ 4 can be expressed as the sum of two primes. This conjecture has recently been verified up to $4 \cdot 10^{11}$ by Sinisalo.

3. *Searching for counter-examples:* We give three well-known examples.

Suppose $\lambda(n) = (-1)^r$, where r is the total number of primes that divide n , counted with multiplicity. Define $L(x) = \sum_{n \leq x} \lambda(n)$. Polya conjectured that $L(x) \leq 0$ for all $x \geq 2$. However in 1960, with the aid of a computer, R. S. Lehman showed that $L(906180359) = 1$.

Euler had conjectured that there was no nontrivial integer solution to $a^5 + b^5 + c^5 + d^5 = e^5$, but in 1967, using a computer, L. J. Lander and T. R. Parkin found the counterexample $27^5 + 84^5 + 110^5 + 133^5 = 144^5$.

If $\mu(n)$ denotes the well-known Möbius function (see Chapter 2), then define $M(x) = \sum_{n \leq x} \mu(n)$. Then Mertens conjectured that $|M(x)| < \sqrt{x}$. However, using extensive computation, Odlyzko and te Riele showed in 1985 that this conjecture is false.

4. *Proofs by enumeration:* Some theorems may require enormous computation for their proofs, and the computer has made this task more feasible. (A classic non-number-theoretic example is the proof of the four-color conjecture by Appel and Haken.) We mention such a theorem of D. H. Lehmer: Every set of 7 consecutive integers greater than 36 contains either a prime or a multiple of a prime greater than 41.

Fermat's "Last Theorem" is the conjecture that there exist no nontrivial integer solutions to $x^n + y^n = z^n$ for $n > 2$. (It is easy to see that it would suffice to prove the conjecture for prime exponents $n = p$.) Using a computer, D. H. Lehmer and E. Lehmer proved the truth of the conjecture for all prime exponents less than 2000. This bound has been improved to 4,000,000 by Buhler, Crandall, Ernvall, and Metsänkylä, using extensive computations. The *first case* of Fermat's "Last Theorem" states that there exist no solutions with $p \nmid xyz$. Using a computer, D. Coppersmith proved that the first case holds for all prime exponents $p \leq 7.568 \times 10^{17}$. (Recently, A. Wiles and R. Taylor proved that Fermat's "Last Theorem" is true.)

Waring conjectured that every positive integer is the sum of 19 fourth powers. This was recently proven by Balasubramanian, Deshouillers, and Dress, using extensive computations.

5. *Study of number-theoretic algorithms:* In this category we are more interested in the design and analysis of algorithms themselves than we are in the outcome of any specific computation. The categories above provide impetus for this study, especially when existing computational methods prove infeasible for problems of interest.

We may refer to the activities in the first four categories as *computational number theory*. The last category is the subject of this book: *algorithmic number theory*.

1.1 Number Theory and Complexity

Algorithmic number theory is interesting because there are problems at every level of complexity. Let us mention some examples, which are discussed more fully in the text:

1. There is no algorithm to decide whether an arbitrary Diophantine equation is solvable. (A *Diophantine equation* is a multivariate polynomial equation with integer coefficients, such as $x^3 + y^3 - z^3 = 3$. It is solvable if there exist *integer* values of the variables $x, y, z, \dots$ satisfying the equation.)
2. Given positive integers a, b, c , determining if there is a positive integer $x < c$ such that $x^2 \equiv a \pmod{b}$ is *NP*-hard. (Roughly, this means it is at least as difficult as many combinatorial problems which are believed to be hard.)
3. Factoring integers is believed to be intractable; for this problem no really fast algorithm is currently known to exist.
4. If a given integer is composite, we can quickly produce a proof of that fact with an algorithm that uses a source of random numbers. However, we do not know how to do this with a fast *deterministic* algorithm.
5. While we can do the basic operations of arithmetic quickly in parallel, there is no known fast parallel algorithm for computing the greatest common divisor.
6. Even the basic algorithms of arithmetic have not been completely analyzed. For example, we can form the product of two n -bit integers in $O(n(\log n)(\log \log n))$ steps on a random access machine (RAM), but the existence of a linear-time algorithm is still an open question.

The interest in algorithmic number theory is not confined to theoretical topics. Clark, Bannon, and Keller applied the discrete logarithm (see Chapter 12) to the problem of designing a fast hardware counter. There have also been proposals for parallelization of arithmetic hardware, based on the Chinese remainder theorem.

Number-theory algorithms form the backbone of computer algebra systems, which have become essential tools of the trade for physicists and engineers. For instance, if one wishes to compute using exact arithmetic, a greatest common divisor algorithm is required to reduce fractions to lowest terms. Less trivially, polynomial factoring algorithms use "local" methods based on finite and p -adic fields to construct factorizations over the integers.

Some of the best methods for the generation of pseudo-random numbers use either modular arithmetic or arithmetic in finite fields. Recently, G. Marsaglia and A. Zaman proposed a method for designing pseudo-random number generators with extremely long period. Determination of the period crucially depends on integer factorization, a problem

which we address in Chapter 11. Truly random numbers are not efficiently predictable, but this is not a property shared by all pseudo-random number generators. Currently, there are fast methods which are easily inferable, and slower methods whose prediction is computationally equivalent to a problem such as integer factorization. These topics are discussed in more detail in Chapter 15.

Finally, cryptology relies increasingly on algebraic or number-theoretic principles. As the algebraist A. A. Albert remarked in 1941, "It would not be an exaggeration to state that abstract cryptography is identical with abstract mathematics." This new "applied number theory" has obvious military uses, and has been vigorously pursued by the U. S. Department of Defense and the National Security Agency. Regarding applications, G. H. Hardy said in 1940,

There is one comforting conclusion which is easy for a real mathematician. Real mathematics has no effects on war. No one has yet discovered any warlike purpose to be served by the theory of numbers . . . and it seems very unlikely that anyone will do so for many years.

But this seems no longer to be true.

1.2 Number Theory and Computation: A Brief History

From earliest times, number theory has been associated with computation. Let us mention some events, without attempting to be inclusive.

The Euclidean algorithm (c. 300 B.C.) to compute the greatest common divisor of two integers has been called the "oldest nontrivial algorithm that has survived to the present day".

Eratosthenes (c. 250 B.C.) devised a "sieve" to create a list of prime numbers between 1 and n . Sieve methods are still being used in algorithmic number theory; for example, see Chapter 9.

The "extended Euclidean algorithm", which, given relatively prime integers x, y , finds integers a, b such that $ax + by = 1$, was apparently first given by Aryabhata in the Sanskrit astronomical work *Aryabhatiya* in the 5th-6th century A.D.

In the 7th century A.D., Brahmagupta gave an algorithm for obtaining new solutions to the equation $x^2 - Ny^2 = 1$ for certain values of N , once a particular solution is known. A method that works for all N was given by Jayadeva in the 11th century A.D.

Two statements by Fermat and Mersenne in the 17th century stimulated much interest in methods for factorization and primality testing. In 1640, Fermat observed that $2^{2^k} + 1$ is a prime for $0 \leq k \leq 4$, and conjectured it was prime for all $k \geq 0$; however, Euler showed in 1732 that 641 is a divisor of $2^{32} + 1$.

In 1644, Mersenne claimed that $2^p - 1$ was prime for $p = 2, 3, 5, 7, 13, 17, 19, 31, 67, 127, 257$, and for no other primes p less than 257. (The numbers $2^p - 1$ were of great interest because of their connection with *perfect numbers*; see Chapter 9 for more details.) It would not become feasible to test $2^{257} - 1$ for primality until almost three centuries later, so it is not surprising that Mersenne's list contained several mistakes: the numbers corresponding to $p = 67$ and $p = 257$ are not prime, and the list omitted the primes corresponding to $p = 61$ and $p = 89$. Numbers of the form $2^p - 1$ became known as *Mersenne numbers*.

Euler proved in 1772 that the Mersenne number $2^{31} - 1 = 2147483647$ was a prime.

In 1776, C. F. Hindenburg realized the sieve of Eratosthenes by using a "strip of thick paper with holes at proper intervals to show the multiples of p , for the successive primes p ".

At the start of the nineteenth century, Gauss discussed the importance of two basic problems from algorithmic number theory in a quotation that has frequently been cited:

The problem of distinguishing prime numbers from composite numbers and of resolving the latter into their prime factors is known to be one of the most important and useful in arithmetic. It has engaged the industry and wisdom of ancient and modern geometers to such an extent that it would be superfluous to discuss the problem at length. Nevertheless we must confess that all methods that have been proposed thus far are either restricted to very special cases or are so laborious and prolix that even for numbers that do not exceed the limits of tables constructed by estimable men, i.e. for numbers that do not yield to artificial methods, they try the patience of even the practiced calculator. And these methods do not apply at all to larger numbers. Even though the tables, which are available to everyone and which we hope will continue to be extended, are indeed sufficient for most ordinary cases, it frequently happens that the trained calculator will be sufficiently rewarded by reducing large numbers to their factors so that it will compensate for the time spent. Further, the dignity of the science itself seems to require that every possible means be explored for the solution of a problem so elegant and so celebrated.

Other writers were even more specific. For example, in 1874, W. S. Jevons said:

Given any two numbers, we may by a simple and infallible process obtain their product, but it is quite another matter when a large number is given to determine its factors. Can the reader say what two numbers multiplied together will produce the number 8,616,460,799? I think it unlikely that anyone but myself will ever know, for they are two large prime numbers, and can only be rediscovered by trying in succession a long series of prime divisors until the right one be fallen upon. The work would probably occupy a good computer for many weeks, but it did not occupy me many minutes to multiply the two factors together.

(Today we might find this statement ironic, for Jevons himself had built a "logical machine" in 1869.)

There were also efforts to define a "good" or "direct" algorithm. For example, H. J. S. Smith said in an 1859 survey of number theory:

The arithmetical solution of a problem should consist in prescribing a finite number of purely arithmetical operations (exempt from all tentative processes), by which all the numbers satisfying the conditions of the problem, and those only, are obtained.

While mechanical calculators were devised in the seventeenth century by Schickard, Pascal, and Leibniz, the eighteenth century marks the first application of mechanical aids to number-theoretic problems. Around 1776, C. F. Hindenburg and A. Felkel independently invented devices to aid in the construction of factor tables. A description of Felkel's machine, made of rods, is contained in the correspondence of J. H. Lambert. Felkel's machines were even offered for sale to the general public, although it seems unlikely there was much demand.

In June 1822, C. Babbage discussed an application of his "Difference Engine":

With this machine I have constructed... a table from the singular formula $x^2 + x + 41$, which comprises amongst its terms so many prime numbers.

His machine could only display results, and not print them; Babbage employed a friend to write down the results. In a letter to Davy the next month, Babbage discussed the speed of his machine:

[The Difference Engine] proceeded to make a table from the formula $x^2 + x + 41$. In the earlier numbers my friend, in writing quickly, rather more than kept pace with the engine; but as soon as four figures were required, the machine was at least equal in speed to the writer.

The letter also mentions a possibility for a future machine:

I have also certain principles by which, if it should be desirable, a table of prime numbers might be made, extending from 0 to ten millions.

In a series of papers beginning in 1876, the French mathematician E. Lucas developed the first fast (today we would say "polynomial time") method of testing Mersenne numbers for primality. He said

I have thus verified, but only once, I admit, that the number $A = 2^{127} - 1$ is a prime number; this number contains thirty-nine digits....

He pointed out that his test was particularly suited for computation using binary arithmetic.

Lucas may have used the "arithmometer", a simple mechanical calculator that could add and subtract, invented by Thomas de Colmar in 1820; he advocated the use of this machine in an 1878 paper. But apparently he was not completely confident of his calculations, for in 1876 he wrote, "It is with the aid of these theorems that I think I have demonstrated that the number $A = 2^{127} - 1$ is prime." By 1887, he was even less sure, for he said of $2^{61} - 1$, "It is the largest prime number currently known...." Lucas wrote extensively on

mechanical aids to computation and was interested in the construction of machines to apply his primality tests. He wrote in 1876:

I have devised the design of a mechanism that would enable one to find prime numbers having a *thousand* digits in the decimal system, and even a lot more.

Together with the French civil engineer H. Genaille, he planned to construct a simple version of this machine. Genaille reported in 1891 (the year of Lucas's untimely death)

The "arithmetical piano" allows one to give a practical application of the method formulated by Mr. E. Lucas... for the verification of large prime numbers. By simply moving a few pegs, the problem of verification of prime numbers of the form $2^n - 1$ is almost always reduced to just several hours' work. This machine, which may succeed in automatically performing calculations of the greatest importance, will one day realize the goal of having a calculating machine that can perform the arithmetic operations on its own.

In 1912, however, A. Gérardin reported that "unfortunately, Mr. Genaille did not construct anything, and because the archives of the author [Lucas] do not contain anything on this subject, we must mourn the loss [of the machine]."

In 1883, the Russian priest I. M. Pervushin proved that $2^{61} - 1$ was prime, contrary to Mersenne's assertion. This was also proved independently by P. Seelhoff in 1886, and J. Hudelot in 1887. The latter used Lucas' test, requiring 54 hours.

In 1903, F. N. Cole factored $2^{67} - 1$. When E. T. Bell, the historian of mathematics, asked Cole how long it had taken him to find the factorization, he replied, "Three years of Sundays." Bell goes on to say

At the October, 1903 meeting in New York of the American Mathematical Society, Cole had a paper on the program with the modest title *On the factorization of large numbers*. When the chairman called on him for his paper, Cole—who was always a man of very few words—walked to the board, and, saying nothing, proceeded to chalk up the arithmetic for raising 2 to the sixty-seventh power. Then he carefully subtracted 1. Without a word he moved over to a clear space on the board and multiplied out, by longhand,

$$193,707,721 \times 761,838,257,287.$$

The two calculations agreed.... For the first and only time on record, an audience of the American Mathematical Society vigorously applauded the author of a paper delivered before it. Cole took his seat without having uttered a word. Nobody asked him a question.

The story is an excellent example of the difference in complexity between *finding* a solution to an equation and *verifying* that a given solution is correct. The issue of the relative difficulty of these two problems has given rise to the theory of $\mathcal{NP}$ -completeness. Nevertheless, today a microcomputer can find Cole's factorization in a few minutes.

We might credit H. C. Pocklington with the first explicit analysis of a number-theoretic algorithm in terms of "naive bit complexity." In a 1910 paper, he gave an algorithm to find

$\text{ord}_p a$, the least positive e such that $a^e \equiv 1 \pmod{p}$, when the prime factorization of $p - 1$ is known, and remarked

We notice that the labour required here is proportional to a power of the logarithm of the modulus, not to the modulus itself or its square root as in the indirect processes, and hence see that in the case of a large modulus the direct process will be much quicker than the indirect.

Perhaps Pocklington also deserves credit as the inventor of the randomized algorithm. In a 1917 paper, he gave an algorithm for computing square roots modulo a prime p which used approximately $(\log p)^3$ steps and observed “the labour varies roughly as the cube of the number of digits in the modulus, and so remains moderate for large moduli.” His algorithm depended on finding a quadratic nonresidue of a special form by trial and error. He must have viewed this use of randomness as slightly unsatisfactory, for he explained in a footnote:

We have to do this [find the nonresidue] by trial, using the Law of Quadratic Reciprocity, which is a defect in the method. But as for each value of u half the values of t are suitable, there should be no difficulty in finding one.

Apparently Pocklington also made use of mechanical aids to computation, for he writes of a method for primality testing in a 1914 paper, “It is also well adapted for use with the arithmometer.”

In 1896, F. W. Lawrence outlined the plan of a sieving machine to aid in integer factorization. In 1912, both M. Kraitchik and A. Gérardin suggested constructing a sieving machine. A simple version of such a machine was built that same year by P. Carissan and later perfected by E. Carissan. It was exhibited at the Exposition de Machines à Calculer in Paris in June 1920. Carissan claimed that it could test 10-digit numbers for primality in between 2 and 15 minutes.

In 1914, T. E. Mason sketched the design of a mechanical device to apply Lucas’ primality test to Mersenne numbers.

In 1925, A. J. C. Cunningham and H. J. Woodall published a set of tables of factorizations of $b^n \pm 1$ for small nonsquare b and n up to high powers. About this time, mechanical calculators with an automatic multiply and divide feature became generally available in the United States. Many investigators, including D. H. Lehmer, M. Kraitchik, and K. R. Isenmenger worked on factorizations using desk calculators during this period.

The Cunningham-Woodall tables have been extended and corrected over the last sixty years; this ongoing work is now called the “Cunningham Project”. In 1983, a new version of these tables was published, with J. Brillhart, D. H. Lehmer, J. L. Selfridge, B. Tuckerman, and S. S. Wagstaff, Jr., as authors; the second edition appeared in 1988. Many improvements in the algorithms for primality testing and factorization were motivated by the desire to fill the gaps in these tables.

Today, factoring a previously unfactored number from the Cunningham table is the real test for any new factoring method. Wagstaff periodically issues updates to the tables and lists of the “most-wanted” numbers. Currently the most wanted factorizations are those of $2^{569} - 1$, $6^{256} + 1$, and $2^{4096} + 1$.

In the 1920's, D. N. Lehmer invented the *factor stencil*, an aid to the factorization of large numbers. For all squarefree integers r with $|r| \leq 250$ a punched card was prepared. Each card had a large number of cells, each of which was assigned a prime p ; a hole was punched if r is a quadratic residue of p . To factor a number N , one obtains several small quadratic residues of N , and then superimposes the corresponding punched cards. If all the holes corresponding to a particular prime are punched, then that prime is a possible divisor of N . Later these stencils were revised and extended by J. D. Elder to seven sets of cards with 800 primes each, allowing one to factor numbers up to 3,033,696,241. A similar stencil method was invented by R. M. Robinson to solve quadratic congruences.

D. H. Lehmer is one of the major figures in algorithmic number theory in the twentieth century—a pioneer in the development of both algorithms and machines. We mention just a few of his accomplishments: In 1926, he and his father, D. N. Lehmer, built a sieving device out of bicycle parts. This was “the first fully automatic machine to be used for factoring and primality testing” and could sieve 60 numbers a second. The machine was improved with the addition of a photoelectric cell in 1932, and could then scan 5000 numbers a second. By 1966, Lehmer had built a delay-line sieve that could scan one million numbers per second.

In 1930, he improved and made rigorous the test of Lucas for the primality of Mersenne numbers.

In July 1946, Lehmer was the first to apply an electronic digital computer (the ENIAC) to a number-theoretic problem. He computed a list of all primes $p < 4538791$ for which the multiplicative order of 2, modulo p , was less than 300. (See Chapter 5 for the definition of order.)

Lehmer initiated modern algorithmic number theory in a 1969 paper, *Computer technology applied to the theory of numbers*. In this paper, he distinguished between those problems that could be solved in polynomial time, such as computing the greatest common divisor, and those for which no polynomial-time algorithm was known, such as factorization. Lehmer was also the first to use parallelism to solve number-theoretic problems.

Let us return to our history of the application of machines to number theory. The first program ever run on a stored-program electronic digital computer was a routine to compute, via brute-force search, the largest prime factor of an integer—this was on the Mark I computer at Manchester University on 21 June 1948. Later, in June 1949, M. H. A. Newman, G. C. Tootill, and T. Kilburn wrote a routine to use Lucas' primality test to test the Mersenne numbers up to $2^{353} - 1$ for primality.

In 1952, A. Ferrier proved that $(2^{148} + 1)/17$ was prime using only a desk calculator. This still holds the record as the largest prime found without the use of a computer.

The first successful attempt to use an electronic computer to find new Mersenne primes was by R. M. Robinson in 1952. He used Lucas' test on the SWAC to discover that $2^p - 1$ was prime for $p = 521, 607, 1279, 2203$, and 2281 . (The previous year, A. M. Turing had made an attempt to find new Mersenne primes, but was unsuccessful.)

J. L. Selfridge found factors of the Fermat numbers $2^{2^{10}} + 1$ and $2^{2^{16}} + 1$ on the SWAC in 1953.

Since 1953, many investigators have used computers to factor numbers from the Cunningham tables, or to test such factors for primality. We now know 33 Mersenne primes, the largest being $2^{859433} - 1$, a number of 258716 decimal digits.

More recently, investigators have used supercomputers and networks of smaller computers to factor numbers. J. A. Davis and D. B. Holdridge factored $(10^{71} - 1)/9$ on a Cray XMP in 1983. K. J. McCurdy and M. C. Wunderlich factored a 64-digit number on the MPP in 1986.

The first to use a distributed implementation of factoring algorithms was R. D. Silverman in 1986. He used the quadratic sieve algorithm (discussed in Chapter 11) on a network of Sun microcomputers at the MITRE corporation. Subsequently, A. K. Lenstra and M. Manasse used a parallel implementation of the quadratic sieve and 400 computers on three continents to split a 102-digit factor of $2^{391} - 1$ in 1988.

In 1992, A. K. Lenstra and D. Bernstein factored $2^{523} - 1$, a 158-digit number using the lattice sieve. In 1994, a team of eleven researchers from Amsterdam and Oregon factored the 162-digit number $(12^{151} - 1)/11$ using the number field sieve.

In 1981, J. W. Smith and Wagstaff designed a special-purpose computer, called the Extended Precision Operand Computer, or EPOC, to perform the continued-fraction factoring algorithm. They could factor a 70-digit number in about a month with this machine. H. Dubner and R. Dubner have designed their own special-purpose 200-chip microcomputer for number-theoretic calculations and have found many large primes, some as big as 7000 digits. W. G. Rudd, D. A. Buell, and D. M. Chiarulli are building a special processor for factoring large numbers, as are C. Pomerance, Smith and R. Tuler.

While computers were being applied to specific numbers, there was also increased interest in algorithm development and more rigorous analysis of algorithms.

In 1968, G. E. Collins popularized a measure of the efficiency of number-theoretic algorithms which we call *naive bit complexity*; this notion is discussed more fully in Chapter 3.

J. Brillhart and M. Morrison introduced the first (heuristic) subexponential-time algorithm for factoring in a 1975 paper. (It had been previously considered by Lehmer and Powers in 1931, but was rejected as unsuitable for hand computation with a desk calcula-

tor.) With it, they were able to factor $2^{2^7} + 1$ in 1970. In 1975, R. Schroepel was the first to provide a heuristic analysis of the method's running time. In 1979, J. D. Dixon gave the first subexponential factoring algorithm that could be rigorously analyzed.

In the past twenty years, the development of algorithms in number theory has seen a flurry of activity—so much so that we cannot summarize the highlights here. This period has seen the increased use of concepts from analytic number theory, algebraic number theory, and algebraic geometry. Some of the appropriate chronology is discussed in Chapters 9–12.

With improvements in machines and algorithms, it is now possible to factor 100-digit numbers and test 1000-digit numbers for primality in a reasonable length of time.

We mention briefly some software packages designed especially for number-theoretic computations. Most systems for computer algebra, such as SAC-2, Macsyma, Maple, Mathematica, and Scratchpad, have routines for unlimited precision integer arithmetic, and so are suitable for simple number-theoretic computations. At Berkeley, P. Weinberger and D. H. Lehmer designed a FORTRAN package for number-theoretic computation. D. Cantor and Morrison at UCLA designed a number-theoretic package in assembly language, intended for use with PL/I. Wunderlich and Selfridge have designed an assembly language package for the IBM 360. More recently, H. Cohen and his collaborators C. Batut, D. Bernardi, and M. Olivier have written a package called PARI which is especially designed to optimize number-theoretic calculations. C. Hollinger and P. Serf developed a computer algebra system focusing on algebraic number theory.

1.3 Condensed History of the Theory of Computation

Having examined some of the history of algorithmic number theory, let us look at some events in the development of the theory of computation that relate to the subject of this book.

Hilbert's dream was to provide a formal procedure that could, in principle, give an algorithmic solution to every well-formulated problem in mathematics. At the International Congress of Mathematicians in 1900, Hilbert gave a celebrated list of problems. The tenth one asked for a general procedure to determine if an arbitrary polynomial equation

$$f(x_1, x_2, \dots, x_n) = 0 \tag{1.1}$$

has a solution in integers. Part of the problem was to formalize what was meant by the term "general procedure".

In 1931, Gödel showed that any formal system strong enough to express statements in elementary number theory must contain statements that are undecidable within that system. In lectures given in 1934, Gödel identified mechanically calculable functions with the functions definable using general recursions (*Gödel's thesis*).

At about the same time, A. Church introduced the λ -calculus, a mechanism for defining functions. He claimed (*Church's thesis*) that the set of λ -definable functions coincides with the set of effectively calculable functions, i.e. those computable in principle by any machine or person, with no limitations on time or space.

In 1936, Turing defined a formal model of computation, now known as the *Turing machine*. *Turing's thesis* says that a function is effectively calculable if and only if it is calculated by a Turing machine. He showed that the *halting problem* for Turing machines (given a Turing machine M and an input w , does M halt on input w ?) is undecidable.

In fact, it can be shown that all these models define the same set of functions. (This is also called "Church's thesis".)

Some of the work of Gödel, Church, and Turing was anticipated by E. Post in the 1920's. In 1944, Post introduced several concepts of future importance in complexity theory. His goal was to study the relative computational difficulty of undecidable problems. To do this, he introduced formal notions of reducibilities between problems, and the idea of a "most difficult" or "complete" problem in a given class. (See Chapter 3.) This work, and that of his successors in the 1950's, set the style for present-day complexity theorists.

In 1964, A. Cobham observed that many functions could be computed in *polynomial time*, that is, time bounded by a polynomial in the size of the input. In 1965, J. Edmonds suggested that the terms "good algorithm" and "polynomial-time algorithm" were synonymous. In 1971, S. Cook defined the class $\mathcal{NP}$, and the notions of $\mathcal{NP}$ -completeness and polynomial-time reducibility. Many natural combinatorial problems were shown to be $\mathcal{NP}$ -complete by Cook and R. M. Karp.

In 1971, M. Davis, Yu. Matiyasevich, H. Putnam, and J. Robinson, using ideas that stem from Gödel and Turing, solved Hilbert's Tenth Problem by showing that a general procedure for determining the solvability of Diophantine equations did not exist. In modern terms, we would say that the solvability of (1.1) is undecidable.

A major development was the formal study of algorithms that used an unbiased source of random numbers ("coin flips"). For reasons that are not completely clear, such algorithms have proved to be especially useful in number theory and algebra. These algorithms were popularized by the 1974 result of R. Solovay and V. Strassen (published 1977). Their algorithm tested the primality of a given integer in polynomial time, with the unusual feature that the algorithm answered correctly if n was prime, but might err with very small probability if n was composite. M. O. Rabin obtained a similar algorithm in 1976, based on the thesis of G. L. Miller. Since then, many randomized algorithms for number-theoretic problems have been devised.

In 1976, W. Diffie and M. Hellman introduced the concept of *public-key cryptography*, where a message is encoded using a publicly available algorithm. Decryption of the message, however, would remain difficult unless a secret key were known. They also introduced

algorithmic number theory into cryptology by constructing a key distribution scheme based on the discrete logarithm. The next year, R. L. Rivest, A. Shamir, and L. M. Adleman showed how to realize a public-key cryptosystem using another problem from number theory. Although many other schemes have been proposed (and some broken) since then, the RSA scheme (named for its inventors) still remains a viable method for sending messages securely. These inventions brought new interest in computational number theory from mathematicians, cryptographers, philosophers, and computer security specialists.

1.4 Notes on Chapter 1

A good discussion of factor tables and tables of primes can be found in Glaisher [1878a], D. N. Lehmer [1909, 1914], and D. H. Lehmer [1941]. Lebon [1907, 1908, 1912, 1914] began the construction of a table of primes up to 10^8 , but apparently this was never completed. Also see Tarry [1907]. Currently, the largest table of primes is that of Bays and Hudson, up to 1.2×10^{12} . See Ribenboim [1988b].

For the verification of Goldbach's conjecture, see Sinisalo [1993].

For some famous conjectures inspired by tables, see Birch and Swinnerton-Dyer [1963, 1965] and H. Cohen and H. W. Lenstra [1984a, 1984b].

Discussion of the counterexamples mentioned in the text can be found in Lehman [1960] (also see Tanaka [1980]); Lander and Parkin [1967a]; and Odlyzko and te Riele [1985]. Elkies [1988] found the solution

$$(A, B, C, D) = (2682440, 15365639, 18796760, 20615673)$$

to $A^4 + B^4 + C^4 = D^4$.

The theorem on the prime divisors of 7 consecutive integers can be found in D. H. Lehmer [1964]. For machine-aided proofs of Fermat's last theorem up to various bounds, see D. H. Lehmer, Lehmer, and Vandiver [1954]; Wagstaff [1978]; Tanner and Wagstaff [1987]; Granville and Monagan [1988]; Tanner and Wagstaff [1989]; Coppersmith [1990]; Buhler, Crandall, and Sompolski [1992]; Buhler, Crandall, Ernvall, and Metsänkylä [1993]. For the recent proof of Fermat's "Last Theorem", see Wiles [1995]; Taylor and Wiles [1995].

For the proof of Waring's conjecture on fourth powers, see Balasubramanian, Deshouillers, and Dress [1986a, 1986b]; Deshouillers and Dress [1992].

D. H. Lehmer [1963] drew a distinction between the various uses of computers applied to number theory: a *result* is a statement about a finite set, such as " $2^{2^5} + 1$ is composite". A *theorem* is a statement about an infinite class. Finally, a *theory* is a collection of results and theorems related in "an aesthetically satisfactory way". Machines can easily be used to obtain results, sometime to prove theorems, but rarely to construct theories. Rather, computers are *tools* to aid mathematicians in the construction of theories. For an example of this, see Richards [1974].

There have been several conferences devoted to the use of computers in number theory. Older examples include Atkin and Birch [1971]; Birkhoff and Hall [1971]. A more recent example is Adleman and Huang [1994].

1.1

For the unsolvability of Diophantine equations, see, e.g., Matiyasevich [1993]. For the NP -completeness of solutions to $x^2 \equiv a \pmod{b}$, see Manders and Adleman [1978].

For the use of the Chinese remainder theorem in hardware, see Svoboda [1962]; Szabo and Tanaka [1967]; and Orton, Peppard, and Tavares [1992]. For the application of the discrete logarithm to hardware, see Clark, Bannon, and Keller [1988].

For the pseudo-random number generator with long periods, see Marsaglia and Zaman [1991].

The quotes in this section come from Albert [1941] and G. Hardy [1940a].

1.2

The standard reference for the history of the theory of numbers is the encyclopedic work of Dickson [1919]. A discussion of the application of machines to factorization and primality testing, covering the period 1925 to date, can be found in Brillhart et al. [1983, 1988]. History of early computing devices can be found in Goldstine [1972] and Randell [1982].

The following surveys discuss the use of computers in number theory: E. Lehmer [1956]; Taussky [1956]; Vandiver [1958]; D. H. Lehmer [1963, 1968, 1969, 1971, 1974]; Swinnerton-Dyer [1967]; Fröberg [1968]; Zimmer [1972]; H. Cohen [1979]; Debnath [1982]; H. Williams [1982a]; Nicolas [1985a]; Churchhouse [1988a, 1988b]; Silverman [1991a].

The remark about the Euclidean algorithm comes from Knuth [1981].

The sieve of Eratosthenes was discussed by Nicomachus (see D'oooge, Robbins and Karpinski [1926]) roughly around 125 A. D..

The work of Aryabhata and Brahmagupta is discussed in Weil [1984].

Fermat's observation about the numbers $2^{2^n} + 1$ can be found in Fermat [1894, p. 206]. Euler's factorization of $2^{2^5} + 1$ is in Euler [1849, p. 2]. The American mental calculator Zerah Colburn claimed to have factored $2^{2^5} + 1$ in 1811, at the age of seven, "by the mere operation of his mind" [1833, p. 38].

For Mersenne's statement, see Mersenne [1644] and Dickson [1919, pp. 12–13]. Barlow [1811, p. 43] wrote that $2^{30}(2^{31} - 1)$ was the "greatest perfect number known at present, and probably the greatest that ever will be discovered; for, as they are merely curious without being useful, it is not likely that any person will attempt to find one beyond it." Archibald [1935], Drake [1971], and Heyworth [1982] have discussed possible reasons for Mersenne's conjecture, and Bateman, Selfridge, and Wagstaff [1989] have put forth a new conjecture that may be more plausible.

The primality of $2^{31} - 1$ is discussed in Euler [1849, p. 584].

For discussion of Hindenburg's use of the sieve of Eratosthenes, see Hindenburg [1776] and Dickson [1919, pp. 349].

For quotes on the difficulty of factoring, see Gauss [1801, Art. 329] and Jevons [1874, pp. 141]. For the logical machine, see Jevons [1870]; Akl [1980]; Gardner [1982]. Smith's work can be found in H. Smith [1859].

For the devices of Hindenburg and Felkel, see Hindenburg [1776]; Dickson [1919, pp. 349]; Glaisher [1878a]; Lambert [1785]; and Stephens and Williams [1990].

The difference engine is discussed in Babbage [1889]. The prime-generating polynomial $x^2 - x + 41$ was mentioned by Euler [1849, p. 584]; Legendre [1798] discussed $x^2 + x + 41$. The relationship between these polynomials and the class number of imaginary quadratic fields was explored by Frobenius [1912]; Rabinovitch [1912]; D. H. Lehmer [1936b]; Szekeres [1974]; and Ayoub and Chowla [1981]. Also see H. Cohn [1980, pp. 155–156]; Fendel [1985]; Ribenboim [1988a]; and Fung and Williams [1990].

Lucas' primality test was given in Lucas [1876a, 1876b, 1876c, 1877a, 1878a, 1878b, 1878d]. He discusses the construction of a machine to carry out his test in Lucas [1876b, 1876d, 1877a]; also see Genaille [1891]. The arithmometer is the subject of Lucas [1878b]. A discussion of Lucas' work and a bibliography can be found in Harkin [1957]. For the comment by Gérardin on Genaille's machine, see Gérardin [1912].

For the primality of $2^{61} - 1$, see Bouniakowsky [1884]; Imchenetzki and Bouniakowsky [1887]; Seelhoff [1886]; and Lucas [1887]. For biographical information about Pervushin, and a discussion of his work on $2^{61} - 1$, see Raik [1953].

For information about the early history of mechanical calculators, see Boys [1901]; Turck [1921]; Couffignal [1933]; Lilley [1942]; Comrie [1946]; and Taton and Flad [1963], just to name a few references.

The story about Cole is taken from Bell [1951]; also see Cole [1903].

Pocklington's work can be found in Pocklington [1910, 1914, 1917]. Galois had suggested [1830] that one should find an irreducible polynomial over a finite field by trial and error. But he was not as specific as Pocklington about the running time of his suggestion.

Lawrence's suggestion appears in Lawrence [1896]. For the sieve machines of Kraitchik and Gérardin, see Gérardin [1912]. The work of Carissan is described in Carissan [1920]; a more accessible reference is d'Ocagne [1922]. For the story of the machine's discovery, see Shallit, Williams, and Morain [1995]. As of this writing, Carissan's machine is located in the Conservatoire National des Arts et Métiers, Paris. Stephens and Williams [1990] discuss the history of sieving.

A device to perform Lucas' test is discussed in Mason [1914a, 1914b].

The original Cunningham tables are Cunningham and Woodall [1925]. The modern Cunningham tables are Brillhart et al. [1983] and [1988].

For information about the factor stencils, see D. N. Lehmer [1925, 1929]. For stencils for solving quadratic congruences, see Robinson [1940].

Lehmer's proof and improvement of Lucas' test can be found in D. H. Lehmer [1930a]. His 1946 computation of the multiplicative order of 2 modulo p is mentioned in Lehmer [1974].

For more information about the bicycle-chain sieve and its descendants, see D. N. Lehmer [1932] and D. H. Lehmer [1928b, 1932b, 1933a, 1934, 1966, 1980]. D. H. Lehmer's collected works have appeared as D. H. Lehmer [1981]. Patterson and Williams [1983] have developed a sieve machine that examines 133 million numbers per second.

Lehmer distinguished between "fast" and "slow" number-theoretic algorithms in D. H. Lehmer [1969]; also see D. H. Lehmer [1971, 1973]. For a discussion of parallelism in number-theoretic computation, see D. H. Lehmer [1976a].

An excellent biography of Alan Turing is Hodges [1983]. He mentions that in 1939 Turing designed, but never completed, a machine that would hunt for zeroes of the Riemann zeta function. The method is discussed in Turing [1945, 1953].

For the first program ever run on a stored-program electronic digital computer, see F. Williams [1975, p. 330] and Campbell-Kelly [1980, p. 134] (both these accounts contain typographical errors). For the unsuccessful search for Mersenne primes, see Hodges [1983, p. 392].

Ferrier's result can be found in Ferrier [1952].

The following are papers on the discovery of Mersenne primes by computer: Reid [1953]; Robinson [1954]; Gillies [1964]; Tuckerman [1971]; Slowinski [1979]; Noll and Nickel [1980]; I. Peterson [1988]; Colquitt and Welsh [1991].

For a discussion of the application of supercomputers to factoring, see J. Davis and Holdridge [1983, 1988], McCurdy and Wunderlich [1987] and Lerner [1988].

For a discussion of distributed implementations of factoring algorithms, see Caron and Silverman [1988].

Special processors for number theory are discussed in J. Smith and Wagstaff [1983]; Rudd, Buell, and Chiarulli [1984]; Dubner and Dubner [1986]; Pomerance, Smith, and Tuler [1988]; and Chudnovsky, Denneau, and Younis [1988].

For the use of microcomputers in number theory research, see Wagstaff [1990].

For software packages for number theory, see Wunderlich and Selfridge [1974]; Buell and Ward [1989]; Hollinger and Serf [1991].

1.3

Hilbert's original list of problems can be found in Hilbert [1900]. An English translation is Hilbert [1902]. The proceedings of a 1974 symposium devoted to Hilbert problems is Browder [1976].

For the work of Gödel, Church, Turing, and Post, see Gödel [1931]; Church [1932, 1933]; Turing [1936]; and Post [1944]. The history surrounding these developments is discussed in Kleene [1981] and Rosser [1984]. See M. Davis [1965] for a collection of papers dealing with undecidable problems.

For polynomial time, see Cobham [1964] and Edmonds [1965]. For $\mathcal{NP}$ -completeness, see Cook [1971] and Karp [1972].

A good exposition of Hilbert's tenth problem can be found in M. Davis [1973]. Also see J. Jones and Matiyasevich [1991].

The following are some early examples of the analysis of randomized algorithms: Rabin [1976, 1980b], Solovay and Strassen [1977].

The RSA scheme was introduced in Rivest, Shamir, and Adleman [1978].

2 Fundamentals of Number Theory

This chapter has three purposes. The first is to recall some of the basic definitions and theorems of elementary number theory. This is done in the first three sections. Since we expect that the reader is familiar with some of this material, we do not always provide proofs, although some proofs are sketched as exercises, and references are given.

The second purpose is to state some results from analysis that can be used to estimate common sums and number-theoretic functions. These estimates will be valuable for bounding the running time of algorithms discussed in future chapters.

Finally, we provide a brief review of some notions from modern algebra that provide a very useful context for thinking about number-theoretic algorithms.

We start by giving some definitions and stating some of the fundamental computational problems in algorithmic number theory. We will see later that some of these problems seem to be “easy,” while others are believed to be “hard.”

2.1 Notation, Definitions, and Some Computational Problems

In this section, we state some of the basic definitions and theorems from elementary number theory. We contrast the statements of these theorems with six classical problems from algorithmic number theory.

We denote the ring of integers by the symbol $\mathbb{Z}$, the rational numbers by the symbol $\mathbb{Q}$, the real numbers by the symbol $\mathbb{R}$, and the complex numbers by the symbol $\mathbb{C}$. If z is a complex number, then $\Re(z)$ and $\Im(z)$ denote its real and imaginary part, respectively.

The *greatest integer* function or *floor* function $\lfloor x \rfloor$ maps every real number x to the greatest integer $\leq x$. Thus $\lfloor \pi \rfloor = 3$. Similarly, the *least integer* function, or *ceiling* function $\lceil x \rceil$ maps every real number x to the least integer $\geq x$. Thus $\lceil \pi \rceil = 4$.

We say that an integer m is a *divisor* of an integer n if there exists a third integer a such that $n = am$. If m is a divisor of n , then we write $m \mid n$. For example, $2 \mid 12$.

We say that d is a *common divisor* of m and n if $d \mid m$ and $d \mid n$. For example, 2 is a common divisor of 12 and 30.

The *greatest common divisor* of two integers m and n is a non-negative integer d that is a common divisor of m and n , such that if e is any common divisor of m and n , then $e \mid d$. We write $d = \gcd(m, n)$. Therefore, for $(m, n) \neq (0, 0)$, we see that $\gcd(m, n)$ is actually the *numerically greatest* common divisor, although it has been defined without reference to magnitude. For example, $6 = \gcd(12, 30)$. We define $\gcd(m, 0) = \gcd(0, m) = |m|$ for all m . We say two integers m, n are *relatively prime* if $\gcd(m, n) = 1$; in this case, we sometimes write $m \perp n$.

We say f is a *common multiple* of m and n if $m \mid f$ and $n \mid f$. The *least common multiple* of m and n is a non-negative integer f such that if e is any common multiple of m and n , then $f \mid e$. We write $f = \text{lcm}(m, n)$. We define $\text{lcm}(m, 0) = \text{lcm}(0, m) = 0$ for all m .

PROBLEM 1 Given two positive integers, compute their greatest common divisor and least common multiple.

Problem 1 is discussed in more detail in Chapter 4.

A *prime* is an integer $p > 1$ whose only positive divisors are 1 and p . A number $n > 1$ which is not a prime is said to be *composite*. For example, 2, 3, 5, 7 are primes, but $91 = 7 \cdot 13$ is composite. The number 1 is neither prime nor composite; it is a *unit*.

THEOREM 2.1.1 There are infinitely many primes.

Proof This famous proof is due to Euclid: Assume there are only finitely many primes, say $p_1, p_2, \dots, p_n$. Then $1 + p_1 p_2 \cdots p_n$ is divisible by a prime different from $p_1, p_2, \dots, p_n$, a contradiction. ■

PROBLEM 2 Given a positive integer, determine whether it is prime.

Problem 2 is discussed in more detail in Chapter 9.

The number of primes less than or equal to x is frequently abbreviated $\pi(x)$. We write

$$\pi(x) = \sum_{p \leq x} 1.$$

(This formula uses a convention, which we will use throughout this book, that summations over the variable p refer to primes only.)

PROBLEM 3 Given an integer x , compute the exact value of $\pi(x)$.

(Note: it is known (see Chapter 8 for more details) that

$$\lim_{x \rightarrow \infty} \frac{\pi(x)}{x / \log x} = 1;$$

this is called the *Prime Number Theorem*. Thus $x / \log x$ gives us a good *approximation* for the value of $\pi(x)$. Problem 3, however, asks us to compute the *exact* value of $\pi(x)$.)

Problem 3 is discussed further in Chapter 9.

THEOREM 2.1.2 (THE FUNDAMENTAL THEOREM OF ARITHMETIC) Every positive integer can be expressed as a product of nontrivial powers of distinct primes

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k},$$

and up to a rearrangement of the factors, this *prime factorization* is unique.

For a proof, see Exercise 6. Note that the integer 1 is given by the empty product of primes.

PROBLEM 4 Given a positive integer n , determine its prime factorization.

Problem 4 is discussed in Chapter 11.

Let n be a positive integer, and let a and b be arbitrary integers. Then we write $a \equiv b \pmod{n}$ when $n \mid a - b$; this notation is read as “ a is congruent to b , modulo n ”. If n is understood, we simply write $a \equiv b$. It is easy to verify that $\equiv$ is an equivalence relation on $\mathbb{Z}$ and hence partitions $\mathbb{Z}$ into a set of n disjoint equivalence classes, which we write as $\mathbb{Z}/(n)$.

Frequently it will be useful to identify a canonical *representative* for each such equivalence class. A simple choice for n positive is the least non-negative residue. We abbreviate this choice as $a \bmod n$; this unparenthesized “mod” is a *function* defined as follows:

$$a \bmod n = \begin{cases} a, & \text{if } n = 0; \\ a - n \lfloor \frac{a}{n} \rfloor, & \text{otherwise.} \end{cases} \quad (2.1)$$

Note that this definition is meaningful for all integers a and n .

THEOREM 2.1.3 (FERMAT) If p is a prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$.

For a proof, see Exercise 13.

PROBLEM 5 Given a, b, n , compute $a^b \bmod n$.

Problem 5 is discussed in Chapter 5.

The *Euler phi function*, $\varphi(n)$, is defined to be the number of positive integers $\leq n$ which are relatively prime to n . For example, $\varphi(10) = 4$. We write

$$\varphi(n) = \sum_{\substack{1 \leq k \leq n \\ \gcd(k, n) = 1}} 1.$$

One application of $\varphi(n)$ is the following theorem, a generalization of Theorem 2.1.3:

THEOREM 2.1.4 (EULER) If n is a positive integer, and $\gcd(a, n) = 1$, then $a^{\varphi(n)} \equiv 1 \pmod{n}$.

For a proof, see Exercise 14.

PROBLEM 6 Given a positive integer n , compute $\varphi(n)$.

We will see later that Problems 1 and 5 admit “fast” solutions, while Problems 3, 4, and 6 seem to be “hard”. The status of Problem 2 is too complicated to summarize quickly, and it is covered more thoroughly in chapter 9.

2.4 Notation: Big-O, Little-o, Big-Omega, Big-Theta

In this section, we introduce some useful notation for expressing the asymptotic growth rates of functions.

DEFINITION 2.4.1 We say that $f(n) = O(g(n))$ if there exist constants $c > 0$ and N such that $f(n) \leq cg(n)$ for all $n \geq N$.

We say $f(n) = o(g(n))$ if $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.

We say $f(n) = \Omega(g(n))$ if there exist constants c, N such that $f(n) \geq cg(n)$ for all $n \geq N$.

We say $f(n) = \Omega_{\infty}(g(n))$ if there exists a constant c such that $f(n) \geq cg(n)$ for infinitely many distinct positive integers n .

We say $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $g(n) = O(f(n))$.

Finally, we say $f(n) \sim g(n)$ if $\lim_{n \rightarrow \infty} f(n)/g(n) = 1$.

Sometimes, we use these symbols with functions of more than one variable, which may take negative arguments. For example, if we say that $f(m, n) = O(|mn|)$, then we mean that there exist c, N such that $f(m, n) \leq c|mn|$ whenever $|m| \geq N$ or $|n| \geq N$.

3

A Survey of Complexity Theory

When we develop algorithms to solve problems in number theory, we find that some problems can be solved by algorithms that run quickly, while others cannot be, at least by any known method.

Complexity theory gives us a precise language in which to make such statements; more exactly, it allows us to quantify an algorithm's use of scarce resources such as time and space. Thus, for example, we could prove statements about the number of steps required by the Euclidean algorithm for the greatest common divisor or the method of solving linear equations by Gaussian elimination.

For a given problem, we can, in principle, consider all possible algorithms for that problem, and compare their use of resources. We may also separate problems into *complexity classes*, which contain those problems that can be solved by *some* algorithm within a given resource bound. By choosing suitable bounds, we obtain many interesting complexity classes.

This chapter introduces the language and models necessary to discuss the complexity of number-theoretic algorithms.

3.1 Notation

Our complexity bounds will be phrased in terms of the size of the input. We define

$$\lg n = \begin{cases} 1, & \text{if } n = 0; \\ 1 + \lfloor \log_2 |n| \rfloor, & \text{if } n \neq 0. \end{cases}$$

Then $\lg n$ counts the number of bits in the binary representation of the integer n , excepting the sign bit. We will also denote the length of a character string x by $\lg x$.

We also recall the asymptotic notations defined in Chapter 2: O , o , and Ω . Roughly speaking, $f = O(g)$ means that f is at most proportional to g , $f = o(g)$ means that f/g tends to 0 for large arguments, and $f = \Omega(g)$ means that f is at least proportional to g . (See Section 2.4.)

3.2 The Notion of "Step"

To discuss the running time of a given algorithm, we need the notion of a *step*—the fundamental unit of computation. As with many other types of measurement, there is no single unit that is natural for all situations. For example, in analyzing sorting algorithms we might want to count the number of comparison steps. Early analyses of the Euclidean algorithm were concerned with bounding the number of "division steps" used. On the other hand, when we discuss calculations such as finding the determinant of a matrix, it seems

natural (and certainly easy) to count each addition, subtraction, multiplication, or division as a single step.

But in these cases, a running time estimate may not correspond at all with what is observed when the algorithm is actually programmed on a computer. One difficulty, which is easy to handle, is the following: different computers have different operation times. For example, one computer might add two 32-bit integers in a microsecond, while another takes a millisecond. For this reason, it is usual to suppress constant factors in stating the running time of an algorithm; for such purposes, “big- O ” notation is natural and convenient.

For example, a statement such as “the ordinary pencil-and-paper algorithm adds two n -bit integers in $O(n)$ steps”, although asymptotic, nevertheless accurately describes the behavior of the algorithm on many different computers.

Big- O notation is useful for two other reasons. First, it lets us ignore small terms that occur in analyzing the running time of algorithms. For example, an algorithm that uses $n^2 + 3n + 2$ steps on an input of size n can be succinctly described as an $O(n^2)$ algorithm. Second, it lets us focus on the behavior of algorithms for large inputs, which is frequently (but not necessarily) a good indication of how they will behave in practice.

Our use of such notation is analogous to the use of quantitative theories by physical scientists: by suppressing details, it is frequently possible to give a crisp account of some phenomenon. However, the price paid for such simplifications is that the results have a limited range of applicability. The same is true of our estimates: it is possible that the details suppressed by a big- O estimate may be more important than the asymptotic result.

For the present purpose, a more important objection to assigning arithmetic operations unit cost is the following one. This assumption becomes unrealistic when the size of the inputs greatly exceeds the fixed word length of the computer, or when the required accuracy of the result demands extended-precision routines.

Further, if we assume that we can perform arithmetic on integers of arbitrary size in a single step, then we get theoretical results that contradict experience. For example, under such an assumption we could factor an integer in time proportional to its length (see Chapter 11). We know of no actual computer code for factoring with this property.

Thus to analyze our algorithms, we would like to find a unit that is both natural (in the sense that it is easy to work with) and practical (in the sense that it leads to accurate results). These criteria suggest that *it is useful to equate “step” with “bit operation”*.

We explain informally what this means. In principle, we deal only with variables that take the values 0 or 1, and perform the following logical operations upon these variables: conjunction ($\wedge$), disjunction ($\vee$) and negation ($\sim$). More complex objects are represented using these logical variables; for example, an integer may be represented in base 2, as a list of bits. The cost of a computation is then the number of logical operations performed. In this model, the size of operands affects the cost of even simple arithmetic operations. For example, $O(n)$ bit operations are needed to add two n -bit integers.

Counting bit operations as primitive is satisfying for both practical and theoretical reasons. Because all real computers are built using boolean circuits, the number of bit operations involved in a computation is clearly tied to the difficulty of its physical realization. It is also true that the actual cost of a multiprecise arithmetic operation is controlled by the input lengths. More importantly for this book, counting bit operations allows us to focus our attention on the cost of the arithmetic operations we use, and lets us ignore, to some extent, the cost of shuffling data. For these reasons, *unless otherwise stated, all statements made about complexity in this book will be phrased in terms of bit operations.*

Of course, real computers are not programmed one bit at a time. Nor would it be a good idea for us to describe all of our algorithms at the level of boolean operations. Rather, it seems preferable to analyze the basic arithmetic operations once and for all and base our estimates on the results of this analyses.

What we will actually do, then, is the following. In each of our algorithms, there will be a group, ring, or field (perhaps more than one) in which computation takes place. We will assign each operation in such a structure a certain cost, based on the number of bit operations involved in that operation. Then the total cost of an algorithm is the sum of the costs of the basic individual, ring, or field operations. Although any assignment of costs in such a scheme must ultimately be based on experience, we will take the ring of integers (with four operations) as basic, and derive costs, based on assumptions about integer arithmetic, for more complicated structures.

Table 3.1 provides our assumptions about the bit complexity of the four fundamental integer arithmetic operations, based on the ordinary pencil-and-paper algorithms. We have also listed the best known results in this regard, which are for a multi-tape Turing machine; we use the abbreviation

$$\mu(m, n) = \begin{cases} m(\lg n)(\lg \lg n), & \text{if } m \geq n; \\ n(\lg m)(\lg \lg m), & \text{otherwise.} \end{cases}$$

In our analyses, we will use the naive bounds, because we believe that results obtained through their use lead to accurate predictions. The reader will not go wrong thinking of

Table 3.1
Cost of Basic Arithmetic Operations

Operation	Naive Bit Complexity	Best Known
$a + b$	$\lg a + \lg b$	$O(\lg a + \lg b)$
$a - b$	$\lg a + \lg b$	$O(\lg a + \lg b)$
ab	$(\lg a)(\lg b)$	$O(\mu(\lg a, \lg b))$
$a = qb + r$	$(\lg q)(\lg b)$	$O(\mu(\lg q, \lg b))$

these bounds as axioms from which running times of more complicated algorithms may be derived.

We also point out one extra assumption that is commonly used, but not often stated: numbers are represented using some base $b \geq 2$, and the individual digits of a number may be accessed at no extra cost. For example, taking $b = 2$, if $n \approx 2^k$, then we can compute $\lfloor n/2^{k/2} \rfloor$, an approximate square root of n , in $O(\lg n)$ steps.

Our assumptions about bit complexity are also naive in the sense that they are not tied to any particular machine model. Using these assumptions, one can reason about algorithms in an informal manner, in much the same way as Church's thesis is used in recursive function theory to simplify the presentation of proofs.

For the moment, we will not present any detailed definition of "algorithm," although we do discuss a formal model for this purpose in Section 3.6. Readers familiar with computability theory may choose their favorite definition of this concept, although they should be warned that our naive assumptions about bit complexity may lead to results that differ slightly from those obtained via a formal model of computation. For readers not familiar with computability theory, we state that by an algorithm, we mean something akin to an ordinary computer program, in which operations may be done on arbitrary integers. By the running time of an algorithm, we mean the total number of bit operations involved in these integer operations, as given above.

3.3 Language Classes

It has also proved quite useful to study complexity theory "in the large," in the sense that running times and similar resource requirements are estimated roughly, certainly less precisely than a practitioner would like. Such a study has the advantage that the resulting theory is relatively insensitive to precise running time assumptions of the sort presented in the last section.

We now indicate the simplifications that are useful for such a study. First, although this book deals with algorithms whose inputs are numbers, we may if we wish consider these inputs to be strings from a finite alphabet. We do this, for example, by assuming that numbers are represented in binary notation. We can also, for the most part, restrict attention to algorithms that solve *decision problems*, that is, algorithms that take a string as input and output a 1 or 0 according to whether or not the string belongs to a particular set. In essence, such an algorithm decides whether or not its input has a specific form.

In this regard, the following definitions are useful. By an *alphabet* we mean any finite set, for example, $\{0, 1\}$. A *language* over a finite alphabet Σ is a set of strings whose symbols are chosen from Σ . We let Σ^* denote the set of all strings over Σ ; thus, a language is just

a subset of Σ^* . For example, PRIMES is a language over the alphabet $\{0, 1\}$ consisting of those strings that are the binary representation of a prime number:

$$\text{PRIMES} = \{10, 11, 101, 111, 1011, 1101, 10001, \dots\}.$$

If an algorithm takes a string as input and returns either 0 or 1, we may think of it as solving a decision problem for a particular language L . More precisely, such an algorithm takes a string x as input, and returns 1 if $x \in L$, and 0 otherwise. In this case, we say that the algorithm *accepts* the language L .

A *complexity class* is a collection of languages. We may rephrase many questions about the existence of certain algorithms as questions about complexity classes. For example, the collection of languages with decision algorithms forms a complexity class, which in mathematical logic is called the class of *recursive* sets. Then the negative solution of Hilbert's tenth problem may be summarized by saying that a certain set of strings encoding solvable Diophantine equations is not recursive.

Most of the complexity classes of interest to us originate in the notion of polynomial time computation. For this, we adopt the following definition.

DEFINITION 3.3.1 For a string x , let $\lg x$ denote the length of x . We say that a function f is *computable in polynomial time* if there is an algorithm to compute f that takes at most $p(\lg x)$ bit operations on input x , for some polynomial p .

The above definition is open to criticism because neither "algorithm" nor "computation step" has been formally defined. We remark, however, that these concepts can be made formal (we do this in Section 3.6), and our definition will be equivalent to the other definitions of polynomial time computation given in the literature. If the reader wishes, the notion of a polynomial-time computable function may be taken as primitive.

DEFINITION 3.3.2 We say that a language L is contained in the complexity class $\mathcal{P}$ if the characteristic function of L is computable in polynomial time.

For example, it is easy to see that the set

$$\text{EVEN NUMBERS} = \{0, 10, 100, 110, 1000, \dots\}$$

of even natural numbers written in binary is contained in $\mathcal{P}$. (See Exercise 11.)

When we say that a set of integers is contained in $\mathcal{P}$, we really mean that a certain language, containing the binary encodings of numbers in that set, is in $\mathcal{P}$. We emphasize this point, because not doing so may lead one to incorrectly assign sets to $\mathcal{P}$ in certain cases. For example, a number n can be tested for primality by trying divisors up to $\sqrt{n}$, but this is not a polynomial time algorithm; its running time is exponential in the *length* of the input.

A very useful concept is that of *nondeterministic algorithm*. This is a decision algorithm that includes instructions allowing it to choose from a finite set of possibilities; it accepts the input if *at least one* sequence of possible choices could cause it to output a 1. (The other choices may cause the algorithm to output a 0, or run forever.) Informally, the running time of a nondeterministic algorithm is the minimum number of steps in any computation that outputs a 1, should one exist.

One may also think of a nondeterministic algorithm as one that first “guesses” a string indicating which choices would cause it to accept the input, and then attempts to “verify” that the guess is correct. The algorithm accepts the input if a correct, verifiable guess is possible. For example, a nondeterministic algorithm to recognize the set

$$\text{COMPOSITES} = \{100, 110, 1000, 1001, \dots\}$$

of composite numbers written in binary may be specified as follows. If the input is x , the algorithm simply “guesses” a divisor d , $1 < d < x$, and then verifies that $d \mid x$.

For this example, and many others, verification of a correct guess is relatively easy. It is useful to have a complexity class containing languages with this property. Informally, a language is in the class $\mathcal{NP}$ if it can be accepted by a nondeterministic algorithm in polynomial time. Formally, we can define $\mathcal{NP}$ as follows:

DEFINITION 3.3.3

$$\begin{aligned} \mathcal{NP} = \{L \mid \exists \text{ a set } S \in \mathcal{P} \text{ and a polynomial } p \text{ such that} \\ x \in L \iff \exists y, \lg y \leq p(\lg x), \text{ and } (x, y) \in S\}. \end{aligned} \quad (3.1)$$

(By $S \in \mathcal{P}$, we mean that strings encoding the pairs in S can be recognized in polynomial time; for one possible coding scheme, see Exercise 12.) The string y should be thought of as a polynomial-length “proof” that $x \in L$, which may be checked by an algorithm that recognizes S . Note that $\mathcal{P} \subset \mathcal{NP}$; unfortunately it is not known if these classes are the same.

If C is a complexity class, we denote the complexity class obtained by complementing each of its members by $\text{co-}C$. The class $\text{co-}\mathcal{NP}$ therefore contains the languages for which membership can be quickly *disproved* by a nondeterministic algorithm. For example, $\text{PRIMES} \in \text{co-}\mathcal{NP}$, because the set of all binary strings is the disjoint union of PRIMES , COMPOSITES , and OTHERS (strings not denoting numbers greater than 1). Because strings in the third category are easy to recognize, our nondeterministic polynomial time algorithm for recognizing composite numbers may be easily extended to one recognizing strings that do not denote a prime.